Beta Testing Document

for

CreditSnap

Version 1.0

Prepared by

Group 7 **Group Name:** Code Commanders

Abeer Gupta	230031	abeergupta23@iitk.ac.in
Mahir Jain	230618	mahirj23@iitk.ac.in
Harsh Wardhan Singh	230453	harshwss23@iitk.ac.in
Arsh Aryan	230203	arsharyan23@iitk.ac.in
Joel Bansal	230509	joelb23@iitk.ac.in
Prakhar Kaushal	230768	prakhark23@iitk.ac.in
Krish Arora	230571	krishar23@iitk.ac.in
Om Bhartiya	230714	ombhartiya23@iitk.ac.in
Archit Kumar Gupta	230186	architkg23@iitk.ac.in
Krish Vijay Tiwari	230573	krishvt23@iitk.ac.in

Course: CS253

Mentor TA: Imran Shaik

Date: 9/04/2026

CONTENTS II
REVISIONS II

1 INTRODUCTION 1

2 LIST OF BUGS 2

3 OVERALL QUALITY OF THE SOFTWARE 4

APPENDIX A - GROUP LOG 5

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Version 1.0	Om Bhartiya Joel Bansal Krish Arora Krish Vijay Tiwari Prakhar Kaushal Abeer Gupta Arsh Aryan Archit Kumar Gupta Mahir Jain Harsh Wardhan Singh Shekhawat	First version of the beta testing document	09/04/2026

Introduction

Software Overview

CreditSnap is a web-based campus dining and debt management platform intended to handle food ordering and credit tracking within a campus environment. The system allows students to browse canteens, view menus, place orders, and manage their outstanding debts, while also providing canteen operators with functionality to manage orders and transactions.

The platform aims to replace existing informal methods of food ordering and credit maintenance with a centralized digital system. By digitizing these processes, it introduces structured tracking of orders, transactions, and debts, allowing users to access order history, view transaction details, and perform payments through the platform.

Key Features:

- User authentication (login/signup with verification)
- Browsing canteens and viewing menus
- Placing and managing food orders
- Viewing and paying debts
- Access to order and transaction history
- Notifications and profile management
- Canteen-side dashboards for order handling and menu management

The software is designed for use by students and campus food vendors, integrating ordering and credit management into a single system. Given the number of interacting components, including user interfaces, backend services, and data storage, the correctness and consistency of these functionalities are critical for reliable operation.

The system involves interactions between multiple user roles and relies on coordinated functioning of different modules such as authentication, ordering, payment handling, and data storage. As a result, inconsistencies in any one component may affect the overall behavior of the platform.

Additionally, the correctness of real-time updates, such as order status changes and transaction records, is important to ensure that users receive accurate and up-to-date information. Any delays, mismatches, or failures in synchronization between the frontend and backend can lead to incorrect system states.

List of Reported Bugs

BUG 1:

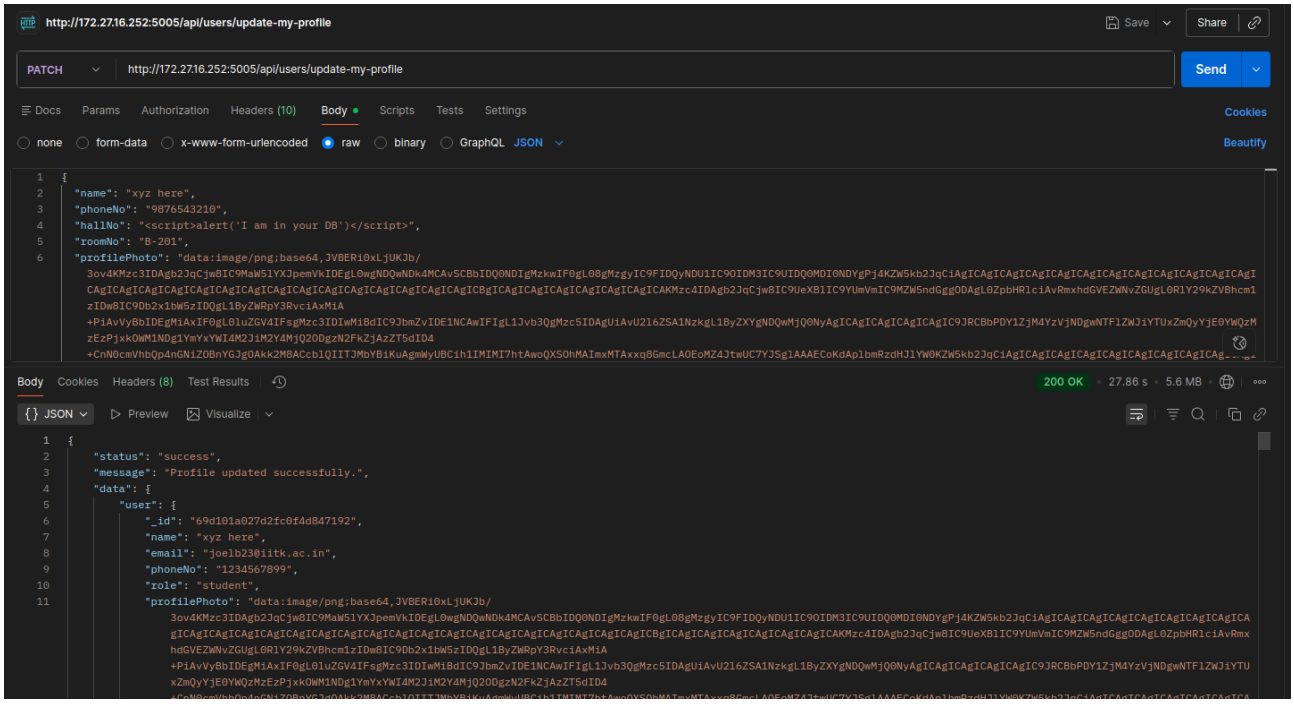
Tested Feature: Profile Picture

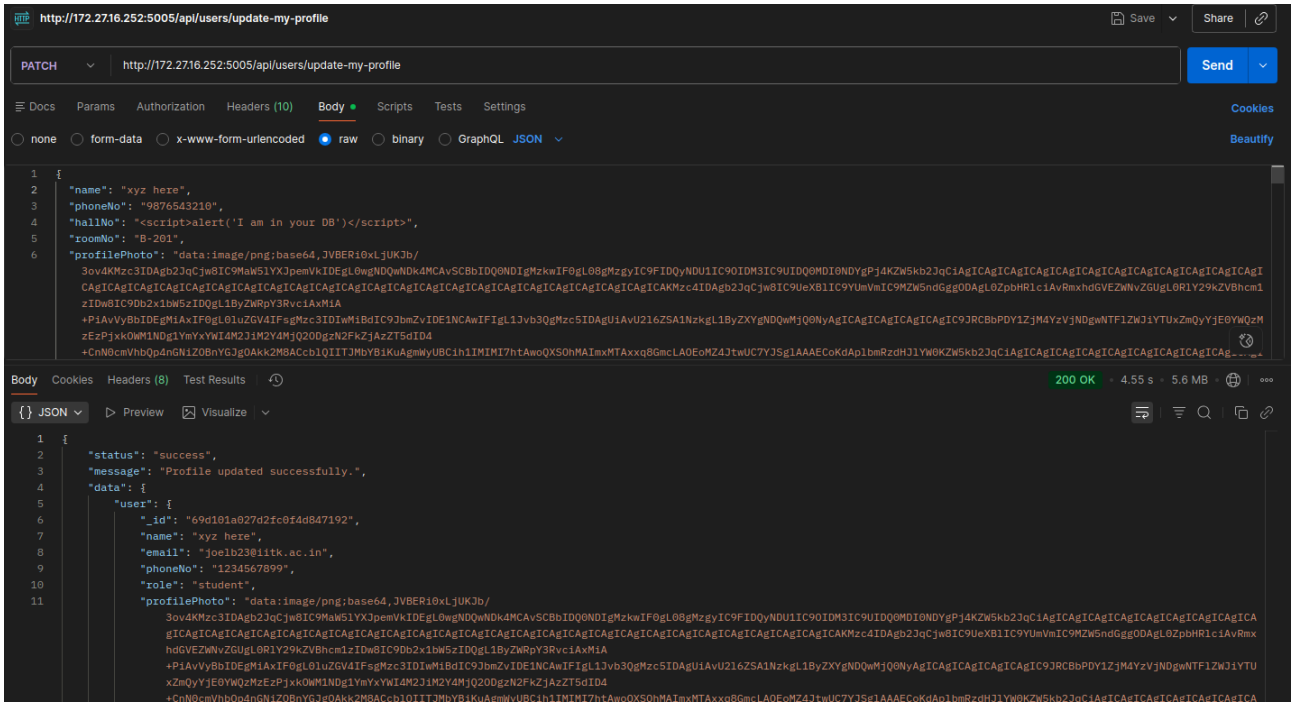
Tester Name: Joel Bansal

Testing Date: 4/04/2026

Bug Details:

The frontend restricts profile picture uploads to a maximum size of 500 KB. However, this restriction is not enforced at the backend. By sending requests directly through Postman, files up to 10 MB can be uploaded successfully, bypassing frontend validation.





Bug Report Date: 9/04/26

Has the bug been fixed? Yes

Date of Bug fixing: 11/04/26

BUG 2:

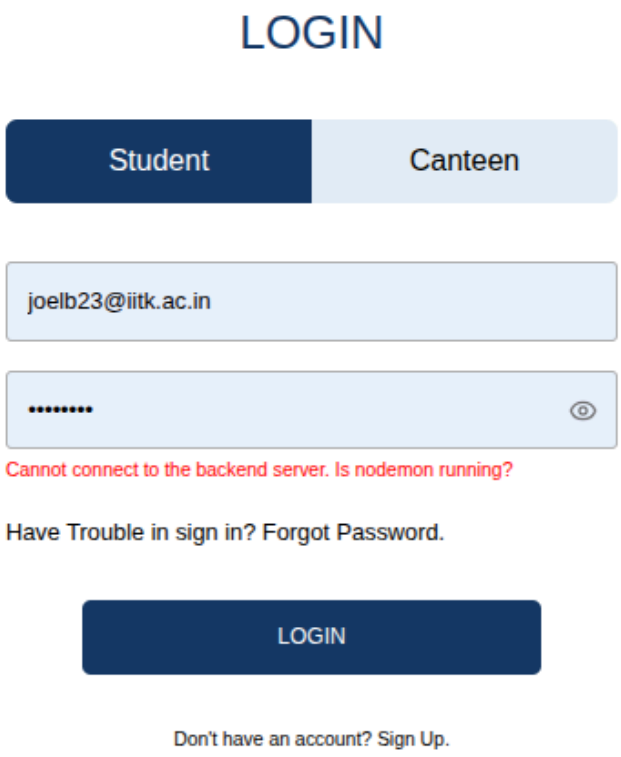
Tested Feature: Backend Connectivity / Server Setup

Tester Name: Joel Bansal

Testing Date: 4/04/26

Bug Details:

The application fails to connect to the backend in certain cases, possibly due to improper server setup or missing instructions regarding running services (e.g., nodemon not running). The issue is not clearly documented, leading to confusion during testing.



Bug Report Date: 9/04/26

Has the bug been fixed? Yes

Date of Bug fixing: 13/04/26

Comment: The issue was not persistent. It only occurred in certain cases. Didn't happen after the issue was pointed out.

BUG 3:

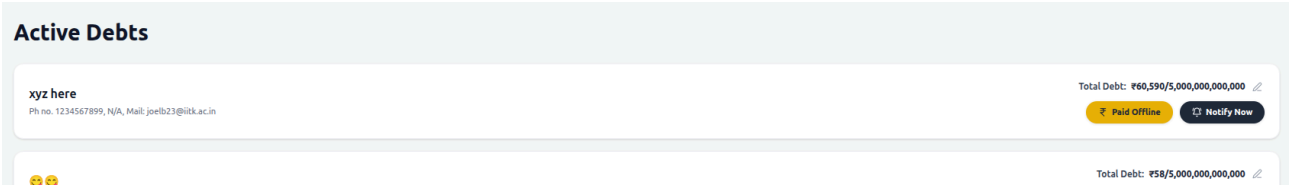
Tested Feature: Order Value Constraints

Tester Name: Joel Bansal

Testing Date: 4/04/26

Bug Details:

When any product in a canteen is priced above ₹3000, the system does not enforce any validation or restriction on the total order value. Orders containing such items are accepted without any further checks, allowing arbitrarily large order amounts to be placed.



Bug Report Date: 9/04/2026

Has the bug been fixed? No

Date of Bug fixing: N/A

BUG 4:

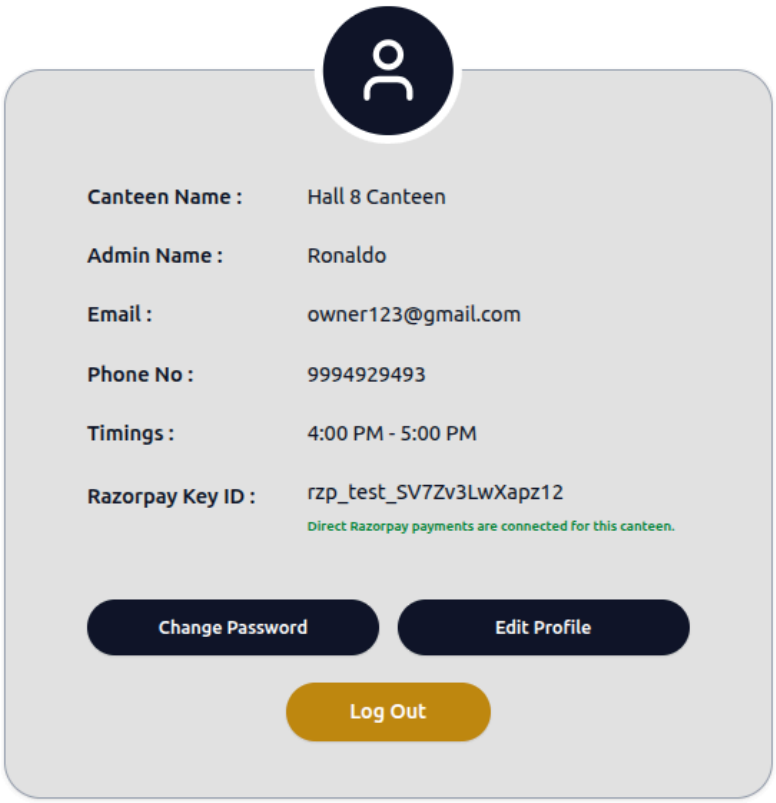
Tested Feature: Canteen Details & Timings.

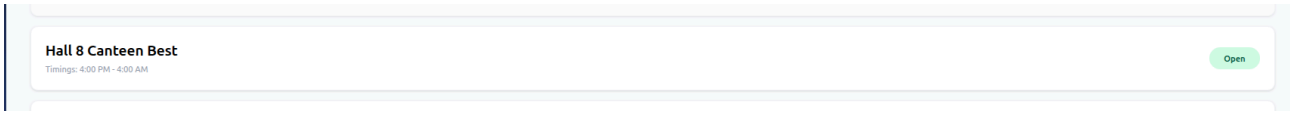
Tester Name: Joel Bansal

Testing Date: 5/04/26

Bug Details:

Canteen timings and names appear to be hardcoded on the frontend. Changes made by canteen owners are not reflected dynamically. Additionally, orders can still be placed even when the canteen is marked closed (e.g., outside operating hours).





Bug Report Date: 9/04/2026

Has the bug been fixed? Partially

Date of Bug fixing: 13/04/26

Comments: Hardcoded frontend logic has been corrected but the orders can still be placed outside of operating hours. If it is the case, the canteen owner has opened the canteen outside of general operating hours, there needs to be a special marker for that which is not the case.

BUG 5:

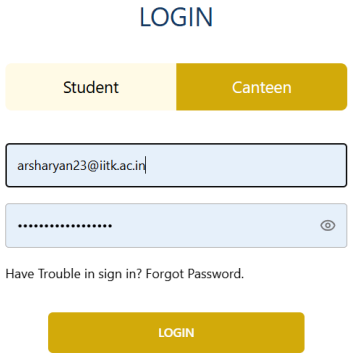
Tested Feature: Role-Based Authentication

Tester Name: Arsh Aryan

Testing Date: 5/04/26

Bug Details:

A student user is able to log in through the canteen login interface. The system does not enforce proper role-based restrictions and should ideally prevent access or throw an error.



Bug Report Date: 9/04/26

Has the bug been fixed? Yes

Date of Bug fixing: 13/04/26

BUG 6:

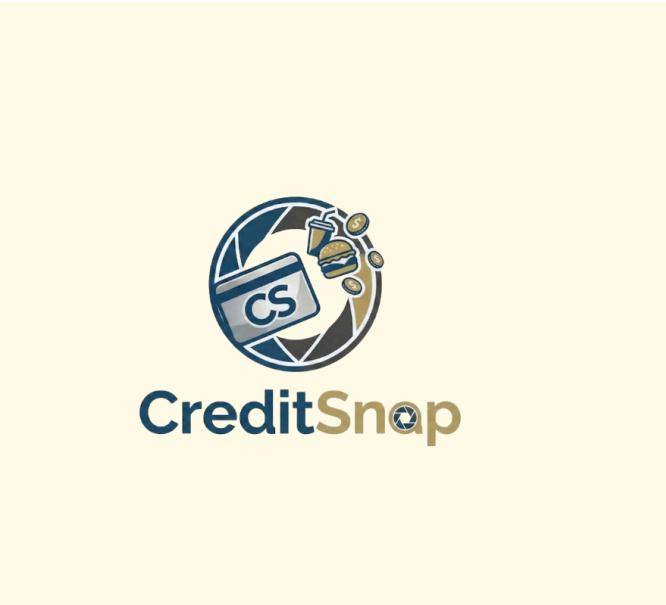
Tested Feature: Canteen Owner Registration

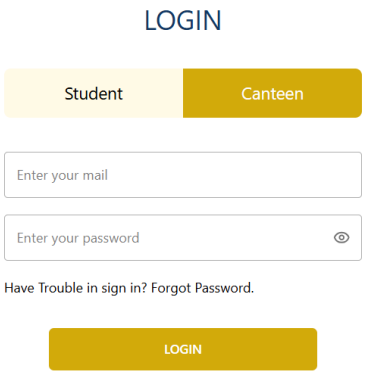
Tester Name: Arsh Aryan

Testing Date: 5/04/26

Bug Details:

There is no option available to sign up as a canteen owner through the platform. This limits usability and restricts onboarding of new canteen operators.





Bug Report Date: 9/04/26

Has the bug been fixed? No

Date of Bug fixing: N/A

Comment: The issue still persists.

BUG 7:

Tested Feature: Project Setup / Installation

Tester Name: Mahir Jain

Testing Date: 08/04/2026

Bug Details:

Following the instructions provided in the GitHub README leads to dependency conflicts during installation, particularly while running npm install in the frontend directory. Additionally, the instructions to run the project are incorrect or incomplete, resulting in failure to properly start the application.

```
C:\Users\User\Documents\sem6\cs253\creditsnap\Credit_Snap\Frontend>npm install
npm error code ERESOLVE
npm error ERESOLVE could not resolve
npm error
npm error While resolving: @tailwindcss/vite@4.2.1
npm error Found: vite@8.0.0
npm error node_modules/vite
npm error   dev vite@"^8.0.0" from the root project
npm error   peer vite@"^8.0.0" from @vitejs/plugin-react@6.0.1
npm error   node_modules/@vitejs/plugin-react
npm error     dev @vitejs/plugin-react@"^6.0.1" from the root project
npm error   2 more (@vitest/mocker, vitest)
npm error
npm error Could not resolve dependency:
npm error peer vite@"^5.2.0 || ^6 || ^7" from @tailwindcss/vite@4.2.1
npm error node_modules/@tailwindcss/vite
npm error   @tailwindcss/vite@"^4.2.1" from the root project
npm error
npm error Conflicting peer dependency: vite@7.3.2
npm error node_modules/vite
npm error   peer vite@"^5.2.0 || ^6 || ^7" from @tailwindcss/vite@4.2.1
npm error   node_modules/@tailwindcss/vite
npm error     @tailwindcss/vite@"^4.2.1" from the root project
npm error
npm error Fix the upstream dependency conflict, or retry
npm error this command with --force or --legacy-peer-deps
npm error to accept an incorrect (and potentially broken) dependency resolution.
npm error
npm error
npm error For a full report see:
npm error C:\Users\User\AppData\Local\npm-cache\_logs\2026-04-08T04_55_35_133Z-eresolve-report.txt
npm error A complete log of this run can be found in: C:\Users\User\AppData\Local\npm-cache\_logs\2026-04-08T04_55_35_133Z-debug-0.log
```

Bug Report Date: 9/04/26

Has the bug been fixed? Yes

Date of Bug fixing: 13/04/26

BUG 8:

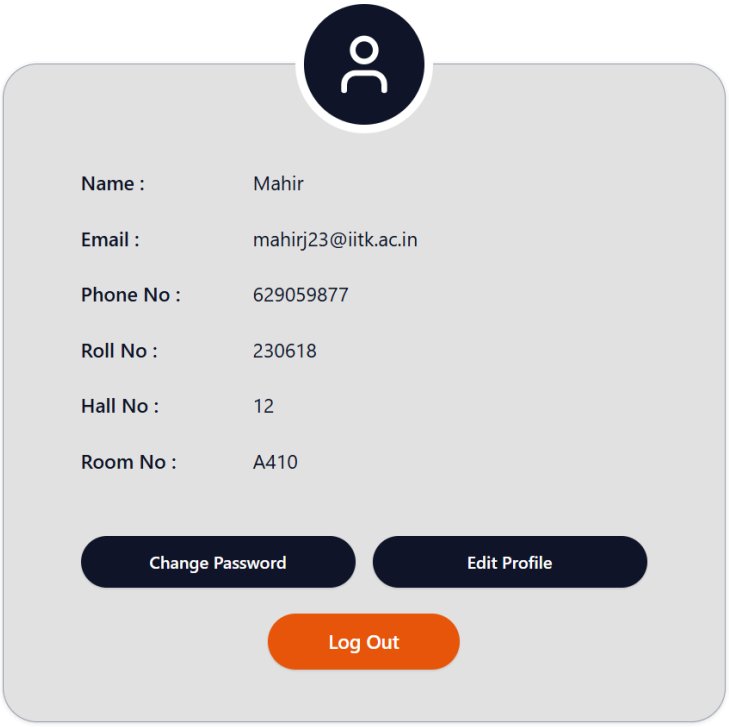
Tested Feature: User Registration (Phone Number Validation)

Tester Name: Mahir Jain

Testing Date: 08/04/2026

Bug Details:

The registration form allows phone numbers with fewer than 10 digits. Proper validation is not enforced, and invalid inputs are accepted without error. Ideally should have thrown an error for the same .



Has the bug been fixed? Yes

Date of Bug fixing: 13/04/26

Comment: Phone numbers can now only be set to 10 digits. Correct indications are given in the process.

BUG 9:

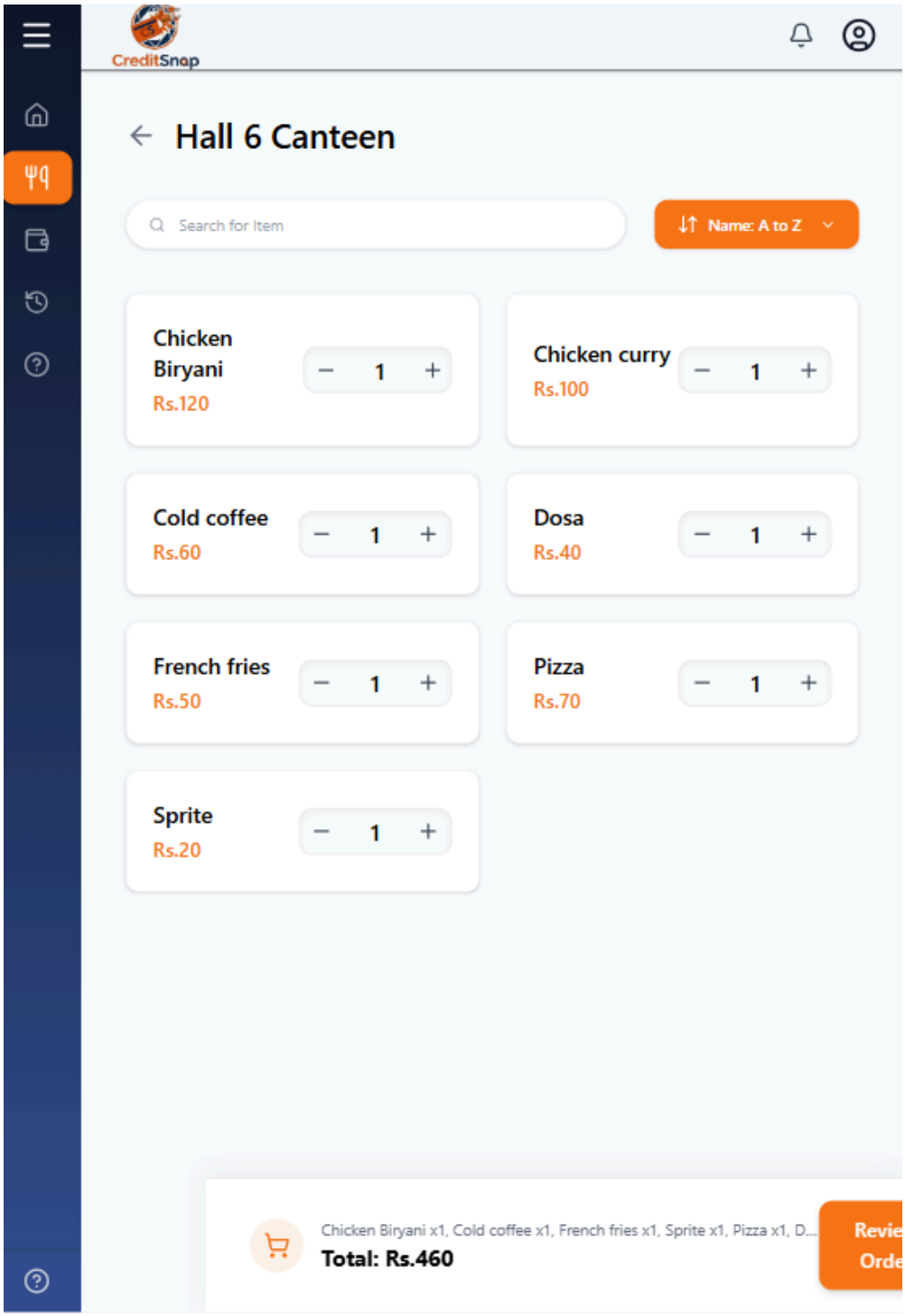
Tested Feature: UI Responsiveness

Tester Name: Mahir Jain

Tested Date: 08/04/2026

Bug Details:

On smaller screen sizes (e.g., iPad Air), adding more than 2–3 items causes the UI layout to break. Elements overlap or become misaligned, reducing usability.



Bug Report Date: 9/04/26

Has the bug been fixed? Yes

Date of Bug fixing: 13/04/26

Comments: Issue was addressed and adding multiple items did not break the responsiveness of the site.

BUG 10:

Tested Feature*:* API Security (CORS Policy)

Tester Name: Mahir Jain

Testing Date: 8/04/2026

Bug Details:

No proper CORS policy is enforced. Requests from any origin can be sent to the server and processed successfully, which may lead to security vulnerabilities.

Code Snippet:

```
const io = new Server(httpServer, {
  cors: {
    origin: (origin, callback) => callback(null, true),  // Allows ANY origin
    methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
    credentials: true,
  },
  // ...
});
```

Bug Report Date: 08/04/2026

Has the bug been fixed? No

Date of Bug fixing: N/A

BUG 11

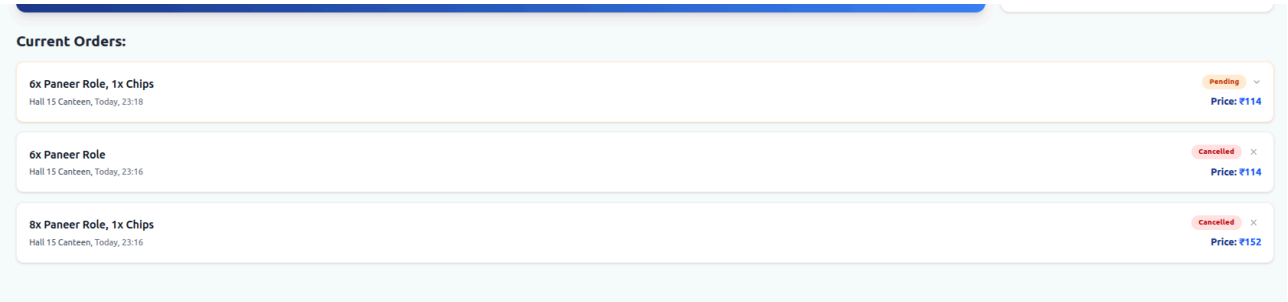
Tested Feature: Order Modification / Editing Orders

Tester Name: Abeer Gupta

Testing Date: 8/04/26

Bug Details:

When attempting to modify or edit an existing order, the system cancels the entire order instead of updating it. Instead of applying the intended changes (such as modifying quantity or items), the order status is directly set to “Cancelled,” resulting in loss of the original order. This behavior indicates that the edit functionality is either incorrectly mapped to the cancel operation or lacks proper handling of update requests.



Bug Report Date: 08/04/2026

Has the bug been fixed? No

Date of Bug fixing: N/A

Comments: The issue still persists.

BUG 12

Tested Feature: Order Acceptance & Debt Synchronization

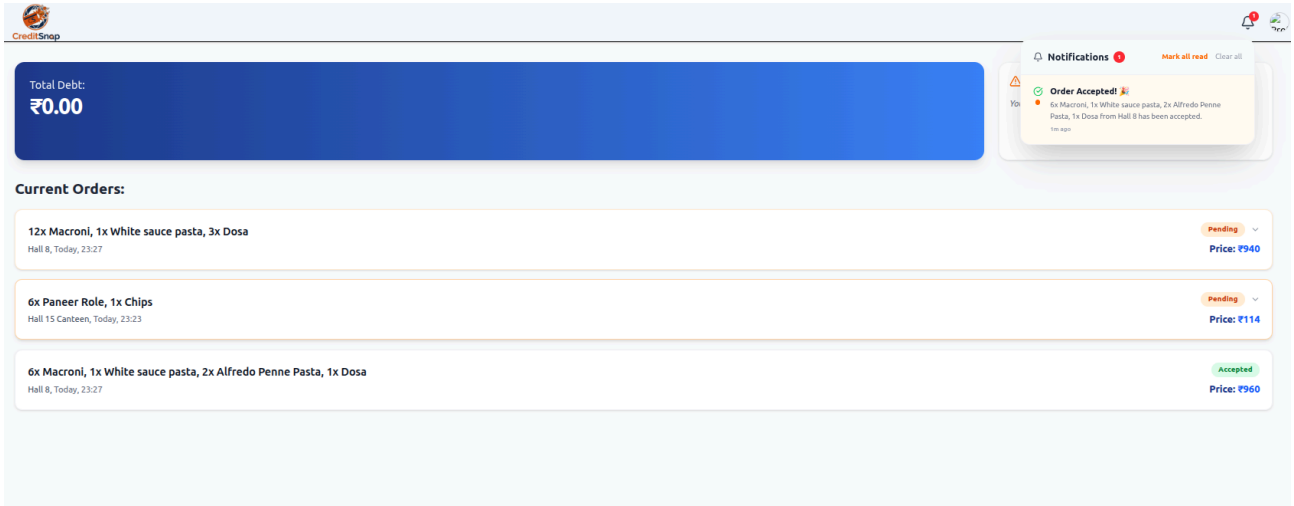
Tester Name: Joel Bansal

Testing Date: 8/04/26

Bug Details:

After placing an order and receiving confirmation that the order has been accepted, the system does not update the user's total debt correctly. The order appears in the "Accepted" state and a notification confirming acceptance is received, but the total debt remains ₹0.00.

This indicates that the debt update logic is not properly triggered or synchronized when an order is accepted. As a result, users can receive accepted orders without any corresponding increase in their recorded debt.



Bug Report Date: 08/04/2026

Has the bug been fixed? Yes

Date of Bug fixing: 13/04/26

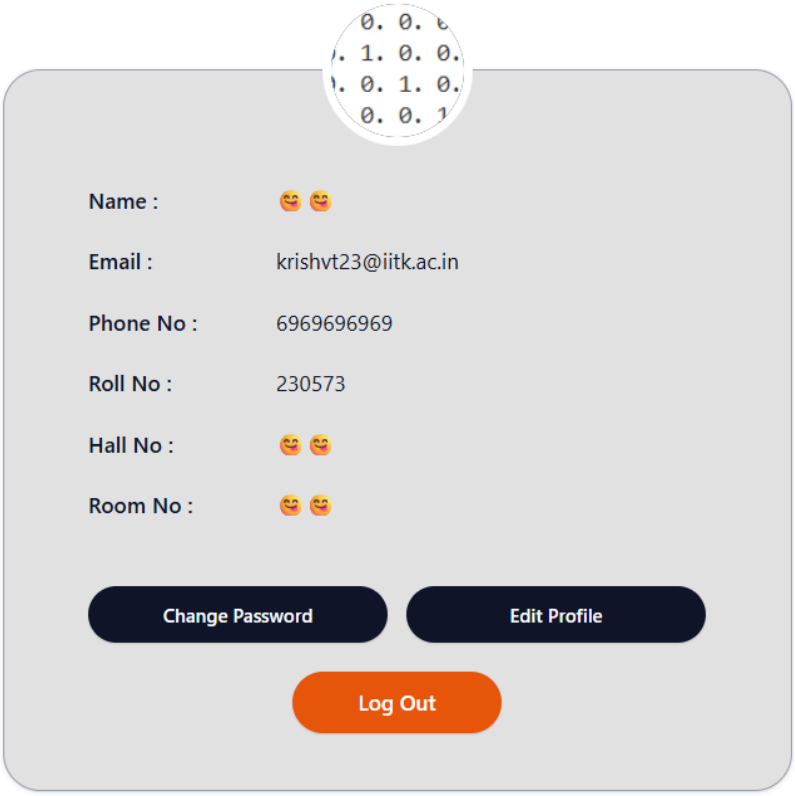
BUG 13

Tested Feature: Name, roll number and room number can be emojis

Tester Name: Krish Vijay Tiwari

Testing Date: 8/04/26

Bug Details: The input fields for user details (Name, Roll Number, and Room Number) lacked proper validation checks. This allowed users to submit emojis and other non-standard characters instead of valid alphanumeric data.



Bug Report Date: 08/04/2026

Has the bug been fixed? Yes

Date of Bug fixing: 10/04/2026

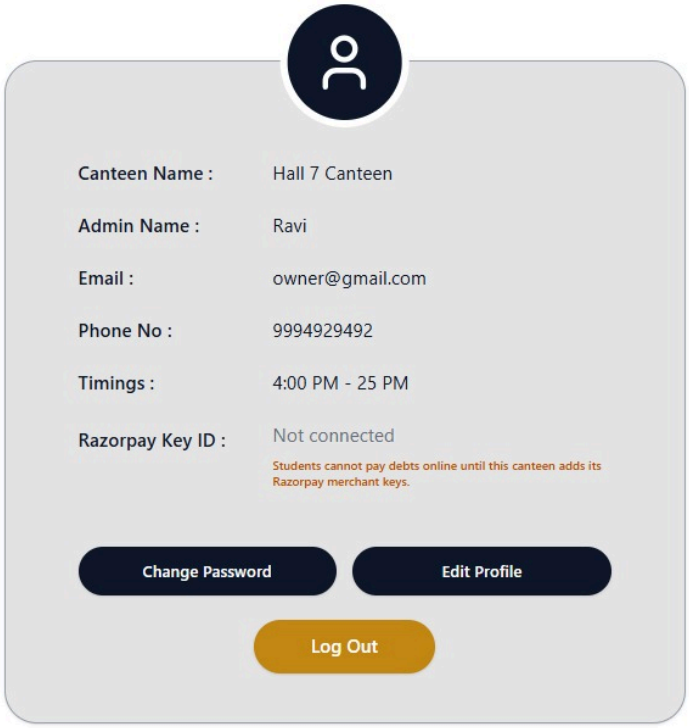
BUG 14

Tested Feature: Canteen Owner Input Validation / Time Field

Tester Name: Harsh

Testing Date: 8/04/26

Bug Details: The time input field on the canteen owner's interface lacked strict data validation. Instead of restricting the input to standard time formats (like HH:MM), the field accepted arbitrary text and invalid characters.



Bug Report Date: 08/04/2026

Has the bug been fixed? Yes

Date of Bug fixing: 10/04/26

BUG 15

Tested Feature: Order API / Backend Validation

Tester Name: Krish Vijay Tiwari

Testing Date: 8/04/26

Bug Details: The `POST /api/orders/place` endpoint was not validating item prices or verifying that the total amount matched the sum of the items multiplied by their quantities. This vulnerability allowed clients to send arbitrary prices and incorrect totals.

Expected Behavior

- The API should validate that each item exists in the menu of the specified canteen.
- If any item is invalid, the API should return an error.

Example:

```
{
  "status": "fail",
  "message": "Item 'Thepla' is not available in this canteen"
}
```

Actual Behavior

- The API successfully creates the order even if the item does not belong to the canteen.

Example response:

```
{
  "status": "success",
  "data": {
    "canteen": "<canteenId>",
    "items": [
      {
        "name": "Thepla",
        "quantity": 1,
        "price": 1
      }
    ],
    "status": "pending"
  }
}
```

Bug Report Date: 08/04/2026

Has the bug been fixed? Yes

Date of Bug fixing: 13/04/26

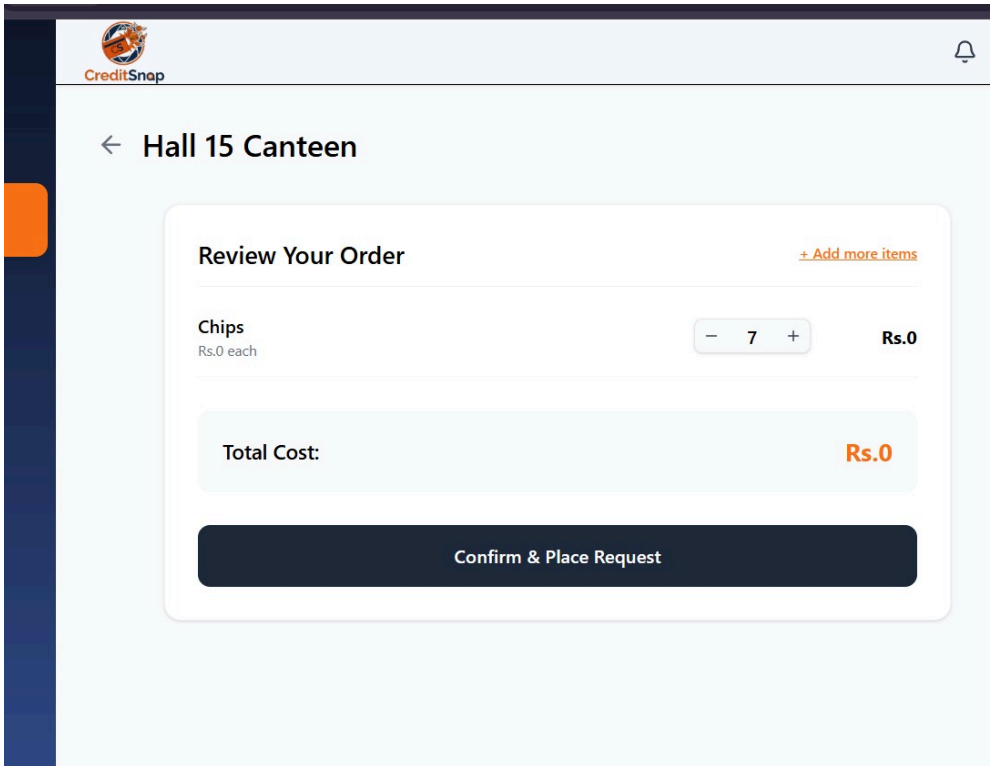
BUG 16

Tested Feature: UI Ordering Logic & Canteen Menu Management

Tester Name: Joel Bansal

Testing Date: 8/04/26

Bug Details: There were two connected issues. First, users saw items flagged as "Available" in the UI, but the order action was completely unresponsive due to a mismatch between the `is_available` status and the `order_validation` logic. Second, canteen owners were able to add food items with a price of Rs.0, which broke the checkout flow.



Bug Report Date: 08/04/2026

Has the bug been fixed? Yes

Date of Bug fixing: 13/04/26

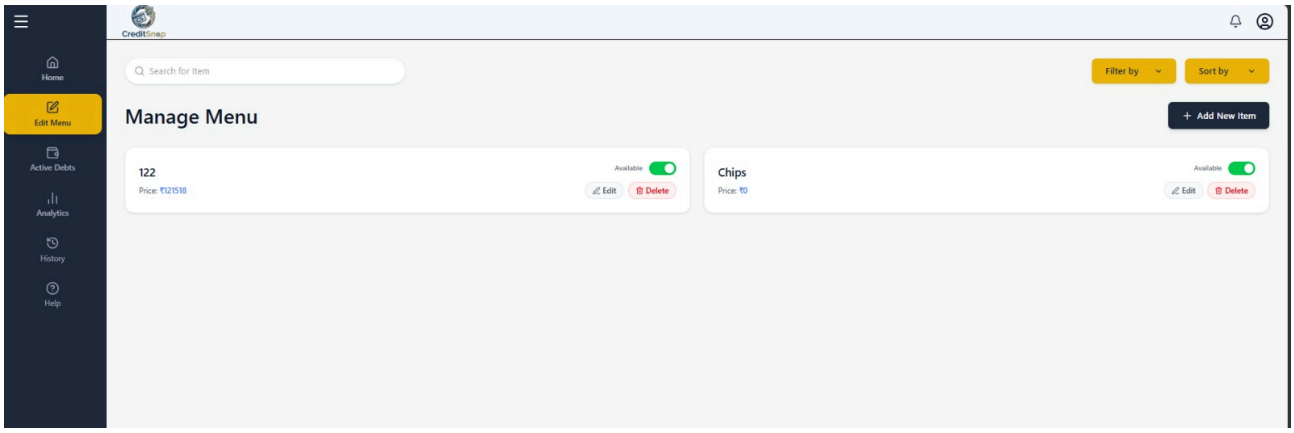
BUG 17

Tested Feature: Menu Management / Input Validation

Tester Name: Abeer Gupta

Testing Date: 8/04/26

Bug Details: The system allowed canteen owners to add new food items to the menu using purely numeric values in the item "Name" field (e.g., naming an item "122").



Bug Report Date: 08/04/2026

Has the bug been fixed? Yes

Date of Bug fixing: 13/04/26

Comments: The field now expects at least one alphabet in the name.

BUG 18

Tested Feature: Security / Authentication Endpoints

Tester Name: Joel Bansal

Testing Date: 8/04/26

Bug Details: The authentication endpoints (like `/api/users/login`) lacked rate limiting. During a stress test with 1,000 concurrent requests, the backend completely crashed and went offline. This vulnerability allowed for potential Denial of Service (DoS) and password Brute Force attacks

stress-test.js > payload > password

```javascript
const axios = require('axios');

const URL = "http://172.27.16.252:5005/api/users/login";
const payload = {
    email: "joelb23@iitk.ac.in", // Use a real or fake email
    password: "bullshit-password"
};

async function fireBurst() {
    console.log("🚀 Starting burst of 1000 requests...");

    const requests = [];
    for (let i = 1; i <= 10; i++) {
        requests.push(
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE

```
joelb23@joelb23:~/Credit_Snap$ node stress-test.js
❌ Req 961: Status FAIL
❌ Req 964: Status FAIL
❌ Req 957: Status FAIL
❌ Req 956: Status FAIL
❌ Req 947: Status FAIL
❌ Req 959: Status FAIL
❌ Req 953: Status FAIL
❌ Req 951: Status FAIL
❌ Req 970: Status FAIL
❌ Req 980: Status FAIL
❌ Req 921: Status FAIL
❌ Req 987: Status FAIL
❌ Req 948: Status FAIL
❌ Req 287: Status FAIL
❌ Req 401: Status FAIL
❌ Req 394: Status FAIL
❌ Req 266: Status FAIL
❌ Req 385: Status FAIL
❌ Req 261: Status FAIL
❌ Req 259: Status FAIL
❌ Req 370: Status FAIL
❌ Req 362: Status FAIL
❌ Req 355: Status FAIL
❌ Req 238: Status FAIL
❌ Req 344: Status FAIL
^C
joelb23@joelb23:~/Credit_Snap$
joelb23@joelb23:~/Credit_Snap$ node stress-test.js
```

Bug Report Date: 08/04/2026

Has the bug been fixed? Partially

Date of Bug fixing: 13/04/26

Comments: Resolved

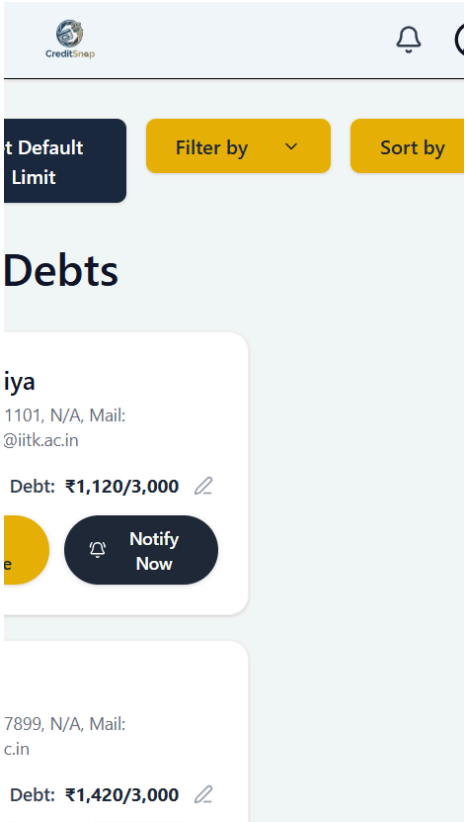
BUG 19

Tested Feature: UI / Frontend Responsiveness

Tester Name: Arsh Aryan

Testing Date: 8/04/26

Bug Details: The UI for the Active Orders Page was not responsive and broke on smaller screen sizes. This was specifically tested and observed on an iPhone SE, where the layout elements overlapped or extended beyond the viewport.



Bug Report Date: 08/04/2026

Has the bug been fixed? Yes

Date of Bug fixing: 13/04/26

Overall Quality of the Software

1. User Manual and Installation Experience

The user manual provides a general overview of the system and its core functionalities; however, it lacks sufficient detail for practical usage. While standard workflows such as login, signup, and order placement are described, the documentation does not adequately cover failure scenarios, incorrect inputs, or recovery from errors.

During testing, issues were encountered while attempting to set up the project locally. Dependency conflicts and incorrect setup instructions resulted in the system not running as expected when following the provided steps. This indicates that the setup process is not robust and may require manual debugging, which is not addressed in the documentation (as also observed in BUG 7).

Overall, while the manual is sufficient for understanding intended functionality, it is not reliable as a standalone guide for installation and troubleshooting.

2. Code Quality and Robustness

The backend follows a structured approach using controllers and route-based modularization, with separation of concerns across features such as users, orders, payments, and debts . Authentication is implemented using JWT-based middleware, which is applied consistently across protected routes.

However, despite this structured design, several issues were observed that indicate gaps in robustness and validation:

- Input validation is inconsistent e.g., invalid phone numbers accepted during registration (like in BUG 8).
- Frontend constraints such as file size limits can be bypassed through direct API requests (BUG 1). This suggests that validation is either incomplete or not enforced uniformly at the backend level.
- Role-based access control is not strictly enforced, as users are able to access functionality outside their intended roles (BUG 5). This indicates insufficient checks in authorization logic.
- Business logic constraints are also not consistently enforced. In certain cases, orders with unrealistic or excessively large values are accepted (BUG 3), and system behavior does not align with expected constraints such as canteen operating hours (BUG 4). These issues suggest gaps in enforcing domain-specific rules.
- The handling of core functionalities such as orders, debts, and payments is distributed across multiple modules. While this modular approach is structurally sound, it increases the risk of inconsistencies if synchronization between components is not carefully maintained.

Overall, while the architectural structure is reasonably organized, the implementation lacks sufficient validation, consistency, and defensive programming practices.

3. Missing Features / Functional Limitations

The system implements the primary functionality for ordering and debt management, but several gaps were identified during testing.

Certain expected features are either missing or incomplete:

- No clear support for canteen owner signup through the public interface (BUG 6) which limits usability and restricts onboarding.
- Role separation between students and canteen operators is not strictly enforced (BUG 5) leading to incorrect system behaviour.
- The system also contains frontend-backend inconsistencies, such as hardcoded canteen details and timings that are not dynamically updated (BUG 4). This affects reliability and creates discrepancies between displayed and actual system state.

Additionally, the system does not appear to handle several real-world scenarios effectively:

- No clear recovery mechanisms for failed or interrupted transactions.
- No safeguards against duplicate or unintended repeated actions.
- Lack of validation for extreme or unrealistic input values in certain cases.

These limitations indicate that while the system handles basic workflows, it may not perform reliably under more complex or edge-case scenarios.

4. Satisfaction of Non-Functional Requirements

From a non-functional perspective, the system shows mixed performance.

- **Reliability:** The presence of inconsistencies between frontend and backend behavior suggests that system state may not always be accurate. Issues such as hardcoded values and validation mismatches contribute to this concern.
- **Security:** While authentication is implemented using JWT, there are notable gaps such as absence of strict CORS enforcement and insufficient input validation, which may allow unintended or unauthorized interactions with the API.
- **Usability:** The interface is generally usable for standard operations; however, UI issues (such as layout breaking on smaller screens) and lack of clear feedback during errors reduce overall usability.
- **Scalability:** The system uses an event-driven backend with real-time updates (e.g., socket-based communication), which suggests potential for scalability. However, lack of strict constraint enforcement and validation may lead to issues under higher load or misuse scenarios .

Overall, while the system demonstrates a functional architecture and basic usability, there are significant concerns regarding robustness, security, and consistency that affect its reliability in real-world deployment.

Appendix A - Group Log

Date/Time	Meet type	What was discussed in the meet?
01/04/26 17:00 - 21:00	Online Meet	Internal testing and preparation of system for upcoming beta testing phase
09/04/26 20:00 - 22:00	Offline Meet (Hall 2)	Discussion of feedback received from beta testing team and categorization of reported issues
10/04/26 16:00 - 19:30	Offline Meet (Hall 2)	Assignment of reported bugs among team members and initiation of fixes
13/04/26 21:00 - 01:00	Offline Meet(Hall 12)	Resolution of reported issues and iterative testing of fixes
16/04/26 21:00 - 01:00	Offline Meet(Hall 12)	Verification of bug fixes, regression testing, and finalization of beta testing report